

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (N_train, 128, 9)

Validation : (N_val, 128, 9)

Testing : (N_test, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

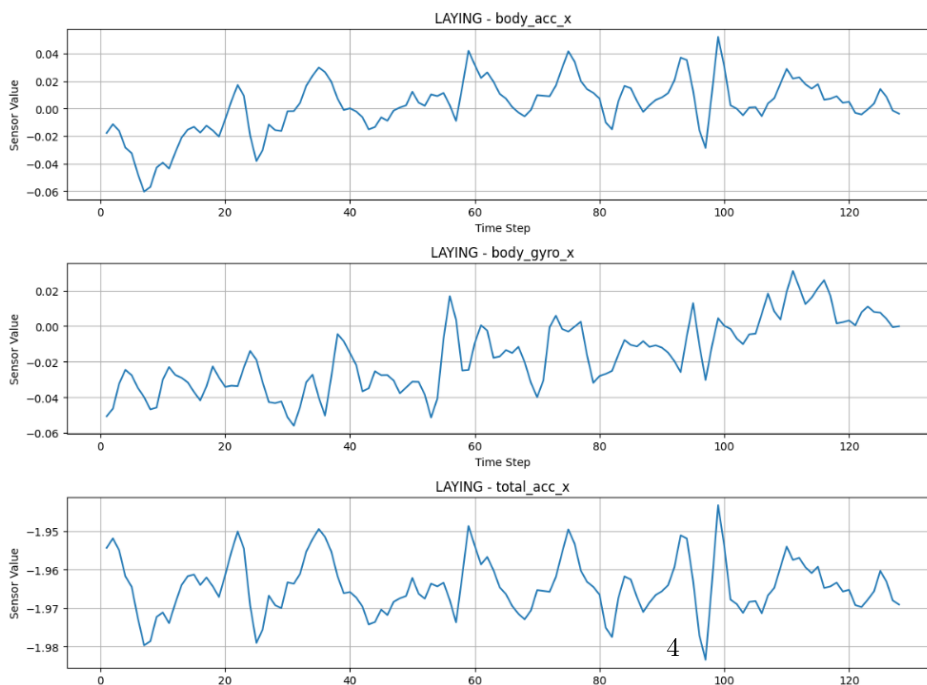
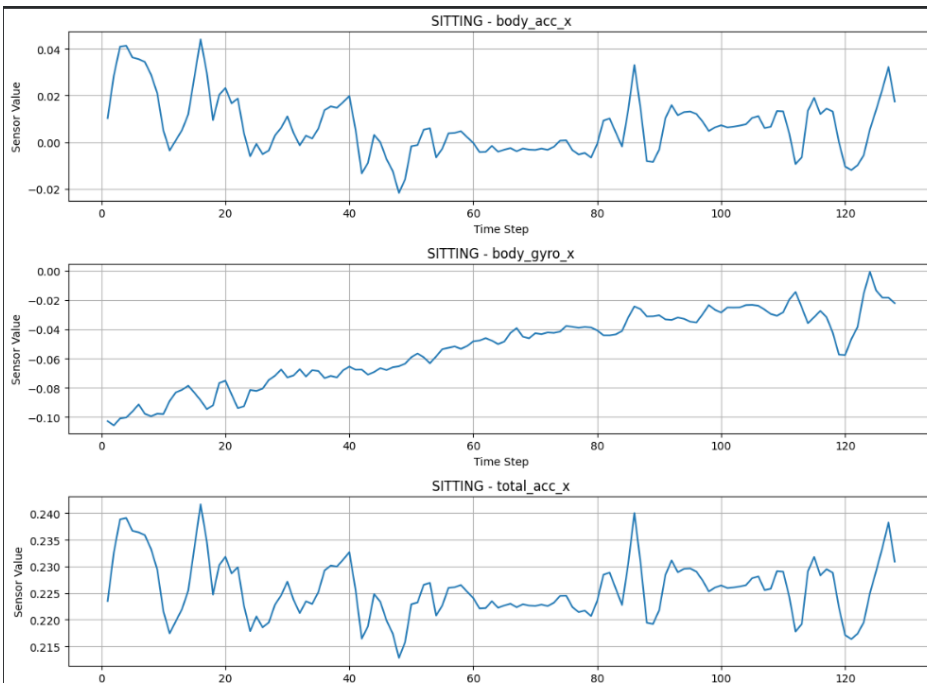
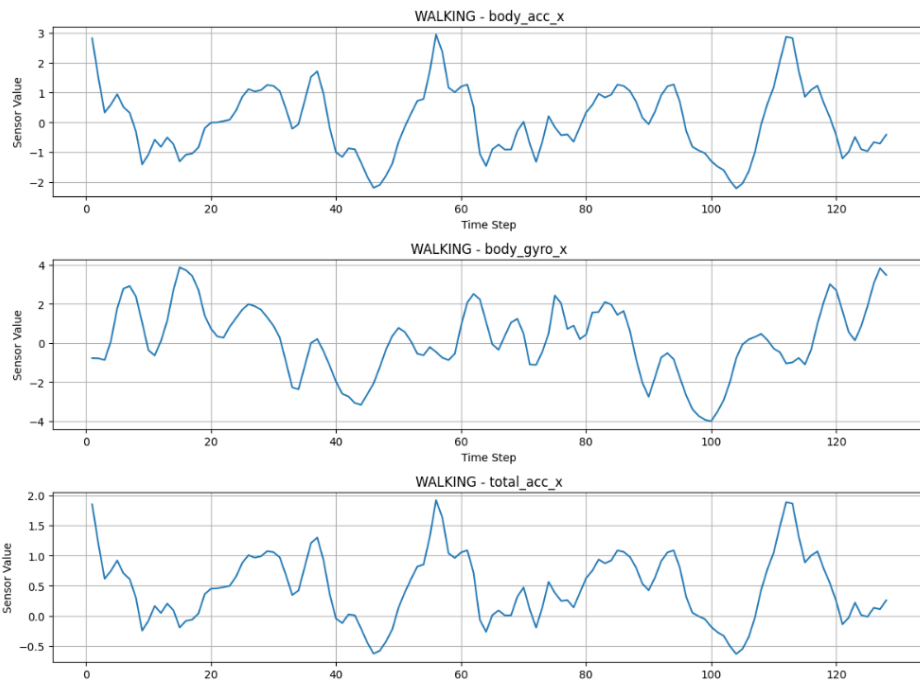
The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.



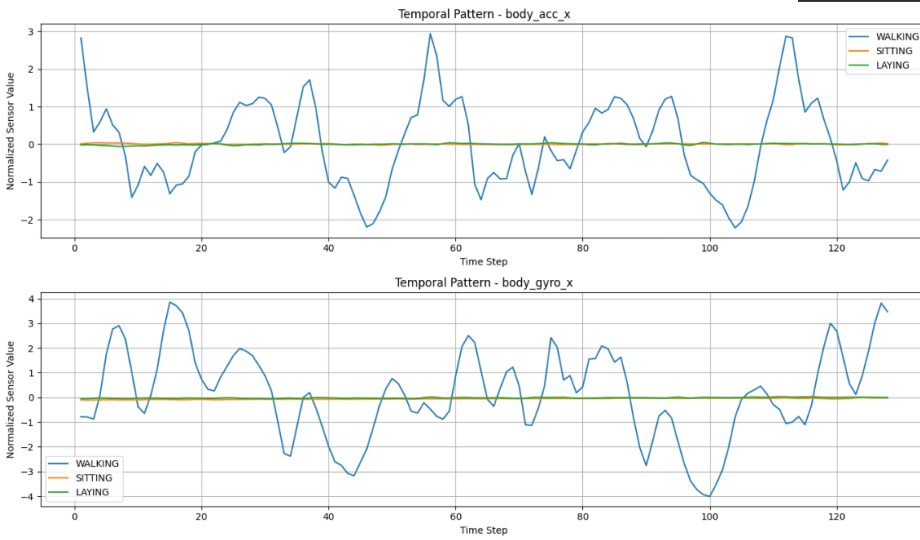


Figure 2: Temporal comparison of `body_acc_x` and `body_gyro_x` for Walking, Sitting, and Laying.

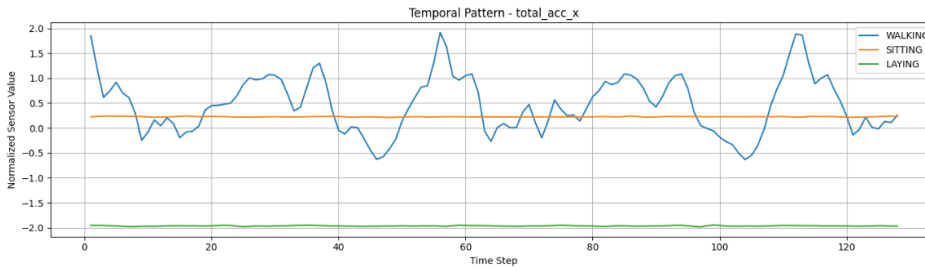


Figure 3: Temporal comparison of `total_acc_x` for Walking, Sitting, and Laying.

Required inference:

1. What temporal pattern is visible?

Walking shows large and frequent fluctuations in the sensor values, with repeated peaks and valleys indicating continuous movement. In contrast, sitting and laying show relatively stable signals with smaller variations over time. Thus, walking has a much stronger temporal pattern than the stationary activities.

2. Which activities exhibit similar patterns?

Sitting and laying exhibit relatively similar patterns because both correspond to stationary activities and have low-amplitude sensor variations. Walking is clearly different, showing much larger and more frequent changes in both accelerometer and gyroscope signals.

3. Why is the ordering of measurements important?

The ordering of measurements preserves the temporal information in the sensor data. Activities such as walking produce characteristic sequences of peaks, valleys, and transitions over time. If the measurements were shuffled, these temporal patterns would be lost, making it difficult for a sequential model such as an RNN, LSTM, or GRU to distinguish between activities.

7. Vanilla RNN Architecture

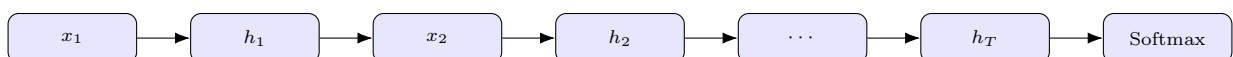
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

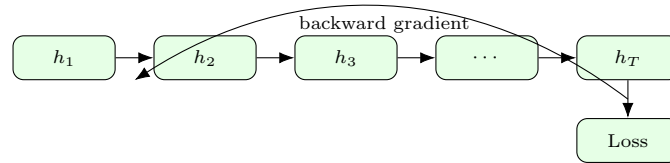
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

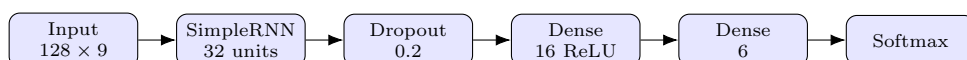
calculate h_1 , h_2 and h_3 manually.

Hidden State	Manual	Program	Difference
h_1	0.336376	0.336376	0.000000
h_2	0.616352	0.616352	0.000000
h_3	0.599958	0.599958	0.000000

The manual and program values are identical up to numerical precision, confirming the correctness of the calculations

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

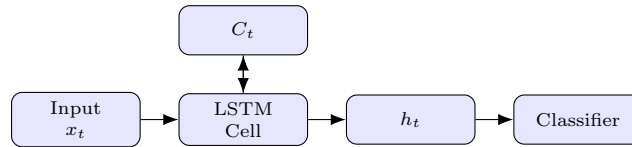
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

Required Inference:

The cell state C_t acts as the long-term memory of the LSTM. It carries important information across multiple time steps while allowing irrelevant information to be removed.

- **Forget gate:** Determines which information from the previous cell state C_{t-1} should be retained or discarded.
- **Input gate:** Determines how much new information from the current input x_t should be stored in the cell state.
- **Candidate cell state:** Provides new information that can be added to the cell state.
- **Cell state:** Combines the retained previous information and the selected new information to maintain memory over time.
- **Output gate:** Controls how much information from the updated cell state is exposed as the hidden state h_t .

Thus, the LSTM selectively retains important information, discards irrelevant information, and incorporates new information at each time step. This controlled flow of information helps the network retain useful information over long sequences and handle long-term dependencies more effectively than a vanilla RNN.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

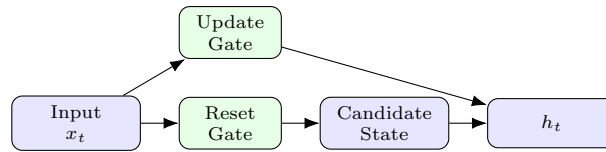
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU. **Inference:** RNN, LSTM, and GRU differ mainly in the way they control and represent information across time. A vanilla RNN has no gates and maintains only a single hidden state h_t , making its architecture the simplest. LSTM introduces three gates—the forget, input, and output gates—and maintains both a cell state C_t and a hidden state h_t , allowing it to preserve important information over longer sequences. GRU simplifies this structure by using two gates—the update and reset gates—while maintaining only a single hidden state h_t .

Model	Gates	States
RNN	No gates	h_t
LSTM	Forget, Input, Output	C_t, h_t
GRU	Update, Reset	h_t

Thus, LSTM has the most explicit memory mechanism, while GRU provides a simpler gated alternative. RNN has the simplest structure, with information represented only through its hidden state.

12. LSTM and GRU Implementation

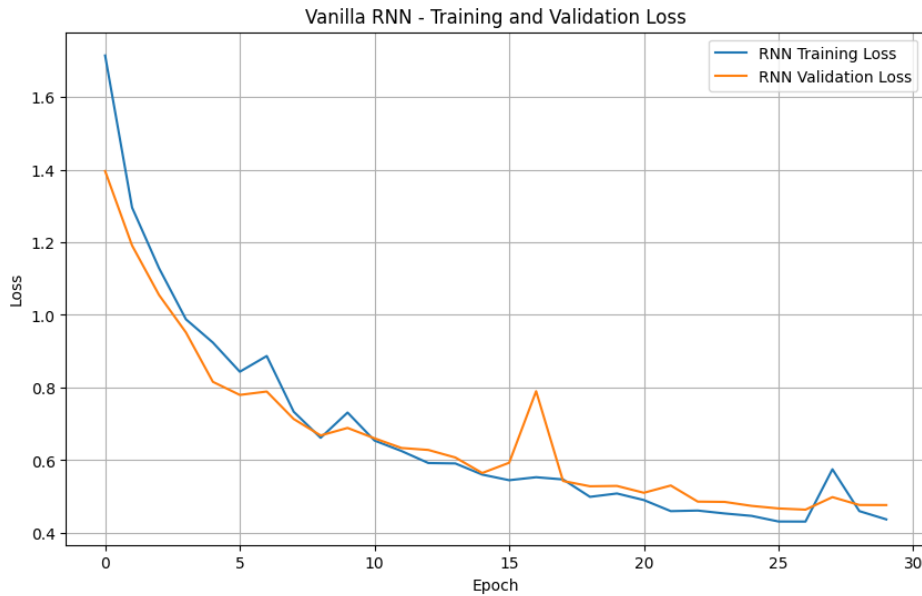
Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

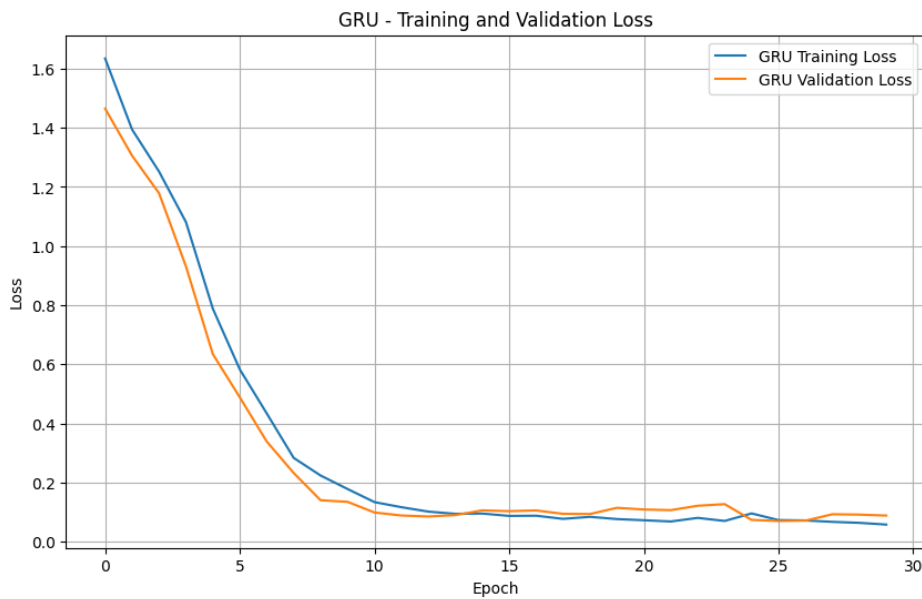
13. Required Training Plots

For each of RNN, LSTM and GRU, generate: **Inference:** Students must identify convergence, overfitting, underfitting and the generalization gap where applicable. **Plot 2: Training and Validation Loss**



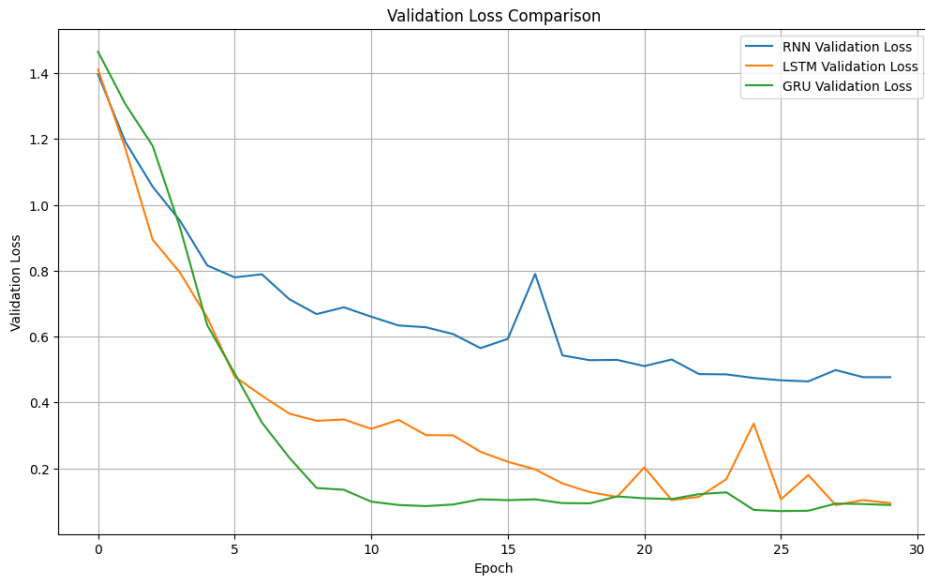
Inference:

- Training and validation loss gradually decrease, indicating convergence.
- The curves remain relatively close in the later epochs, showing a small generalization gap.
- No significant or persistent overfitting is observed.



Inference – LSTM:

- Training and validation loss decrease steadily, indicating good convergence.
- The curves remain close for most epochs, showing a small generalization gap.
- Minor fluctuations are observed in validation loss, but there is no significant persistent overfitting.

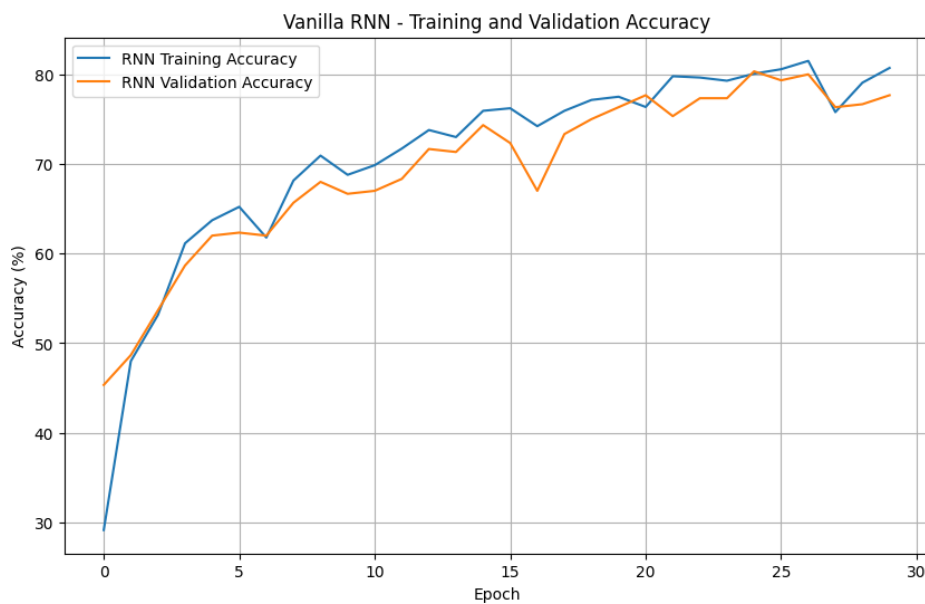


Inference – GRU:

- Training and validation loss decrease rapidly and then stabilize, indicating fast convergence.
- The curves remain very close in the later epochs, showing a small generalization gap.
- No significant overfitting is observed, as both losses remain low and stable.

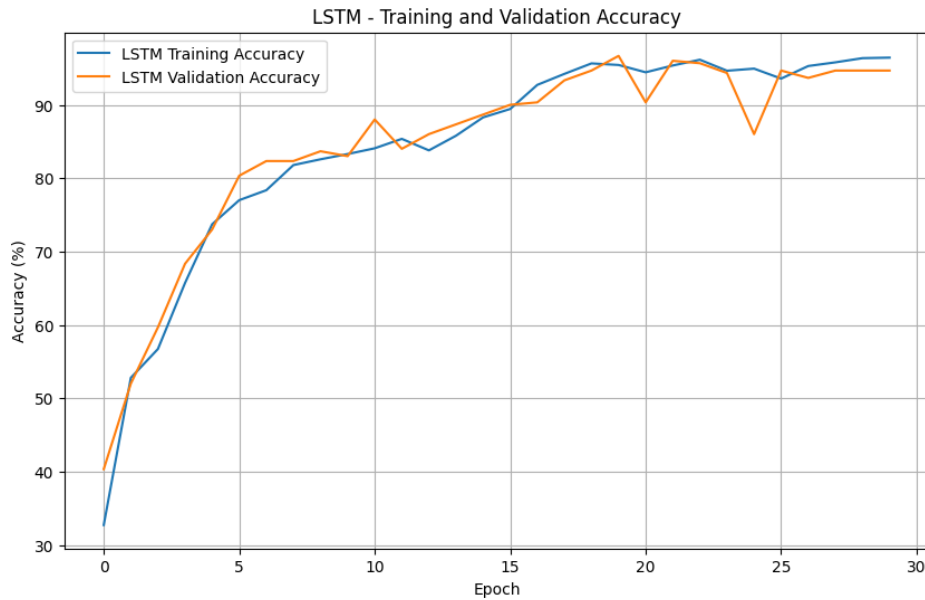
Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy



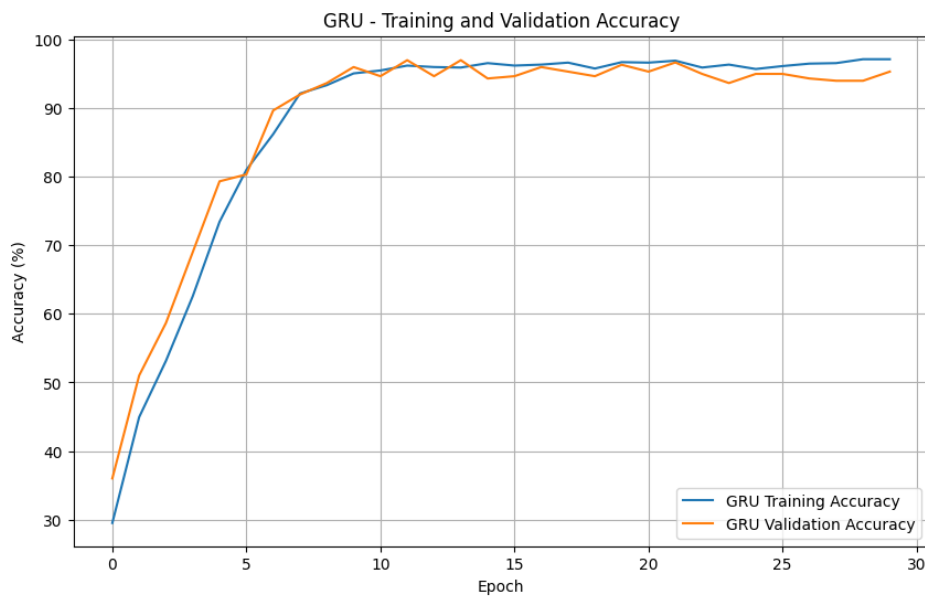
Inference – Vanilla RNN:

- Training and validation accuracy gradually increase, indicating convergence.
- A small gap of about 3% is observed at the end, indicating a small generalization gap.
- No significant overfitting or underfitting is observed.



Inference – LSTM:

- Training and validation accuracy increase and stabilize, indicating good convergence.
- The final accuracy gap is small, around 2%, indicating good generalization.
- No significant overfitting or underfitting is observed.



Inference – GRU:

- Training and validation accuracy rise rapidly and stabilize, showing fast convergence.
- The final accuracy gap is small, around 2%, indicating good generalization.
- No significant overfitting or underfitting is observed.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score

- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Training Accuracy (%)	90.93	97.07	96.57
Validation Accuracy (%)	86.00	95.67	95.33
Parameters	1,974	6,006	4,758
Training Time (s)	38.05	71.18	107.16

15. Confusion Matrix Analysis

Vanilla RNN:

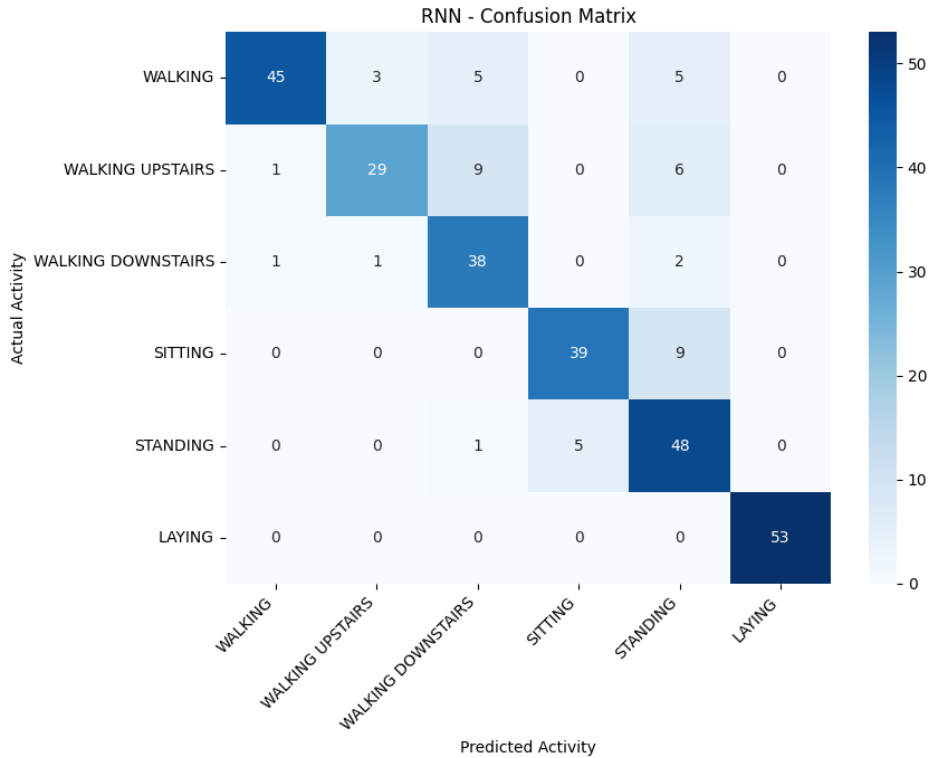


Figure 4: Vanilla RNN Confusion Matrix

Inference – Vanilla RNN:

- The activity with the highest recognition rate is **LAYING**, with a recognition rate of 100.00%.
- The activity with the largest number of misclassifications is **WALKING DOWNSTAIRS**, with 12 misclassified samples.
- The most frequent confusion is **SITTING** → **STANDING**, with 8 samples.
- Other frequent confusions include **WALKING DOWNSTAIRS** → **WALKING UPSTAIRS** (6 samples), **WALKING UPSTAIRS** → **WALKING** (5 samples), and **STANDING** → **SITTING** (5 samples).

LSTM:

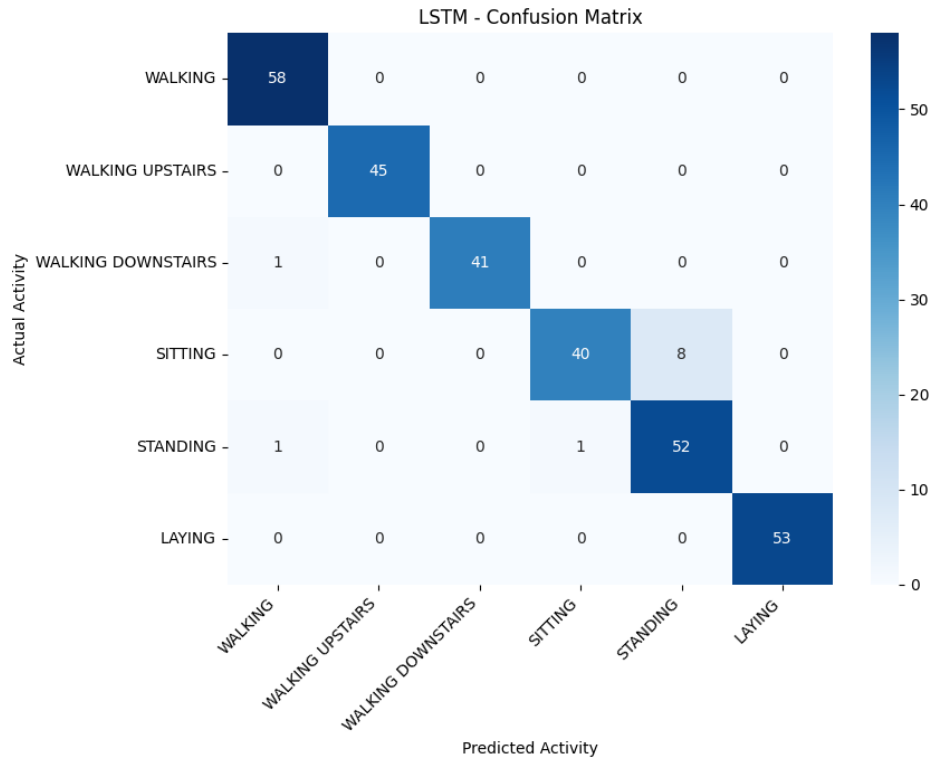


Figure 5: LSTM Confusion Matrix

Inference – LSTM:

- The activities with the highest recognition rate are **WALKING UPSTAIRS** and **LAYING**, both with a recognition rate of 100.00%.
- The activity with the largest number of misclassifications is **STANDING**, with 6 misclassified samples.
- The most frequent confusion is **STANDING** → **SITTING**, with 6 samples.
- The reverse confusion, **SITTING** → **STANDING**, occurs for 5 samples.

GRU:

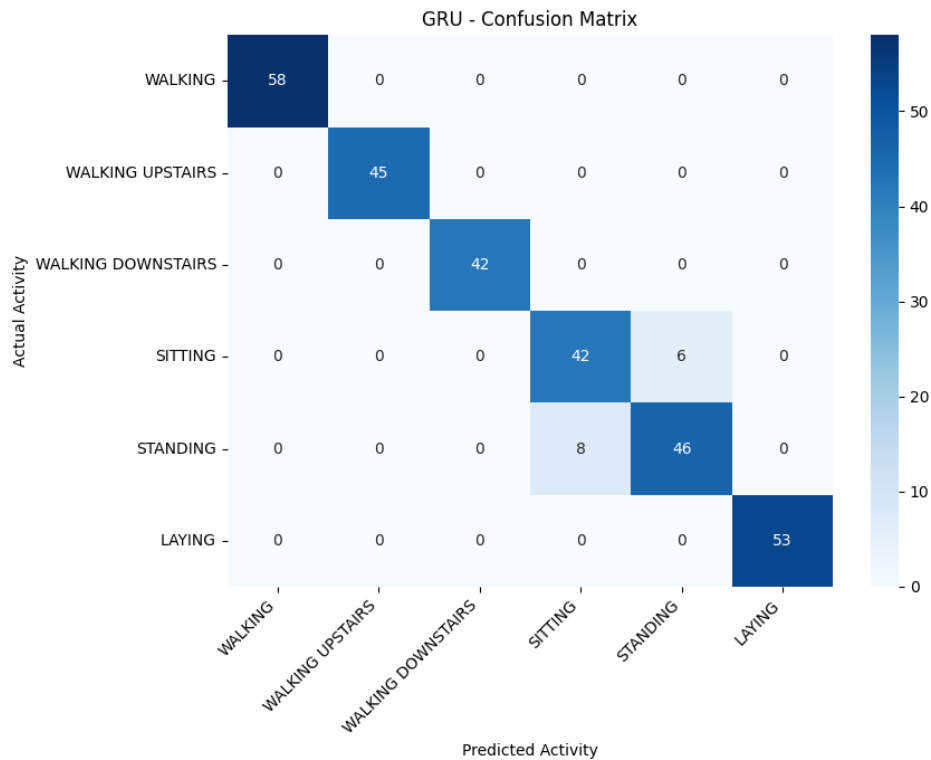


Figure 6: GRU Confusion Matrix

Inference – GRU:

- The activities **WALKING**, **WALKING UPSTAIRS**, **WALKING DOWNSTAIRS**, and **LAYING** have a recognition rate of 100.00%.
- The activity with the largest number of misclassifications is **SITTING**, with 7 misclassified samples.
- The most frequent confusion is **SITTING** → **STANDING**, with 7 samples.
- The reverse confusion, **STANDING** → **SITTING**, occurs for 6 samples.

Cross-Model Inference:

- The **SITTING–STANDING** pair is a recurring source of confusion across all three models.
- For the RNN, the largest number of errors occurs for **WALKING DOWNSTAIRS**, while for LSTM and GRU, the largest errors occur for **STANDING** and **SITTING** respectively.
- The RNN shows additional confusion among the walking activities, particularly **WALKING DOWNSTAIRS** and **WALKING UPSTAIRS**.
- LSTM and GRU substantially reduce the confusion among the walking activities compared with the RNN.
- The error pattern is therefore **partially consistent** across the three models: **SITTING** and **STANDING** are consistently confused, while the activity with the largest number of errors differs between models.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	38.05	71.18	107.16
Test F1-score (%)	85.76	95.38	95.73

Plot 5: Model Performance Comparison

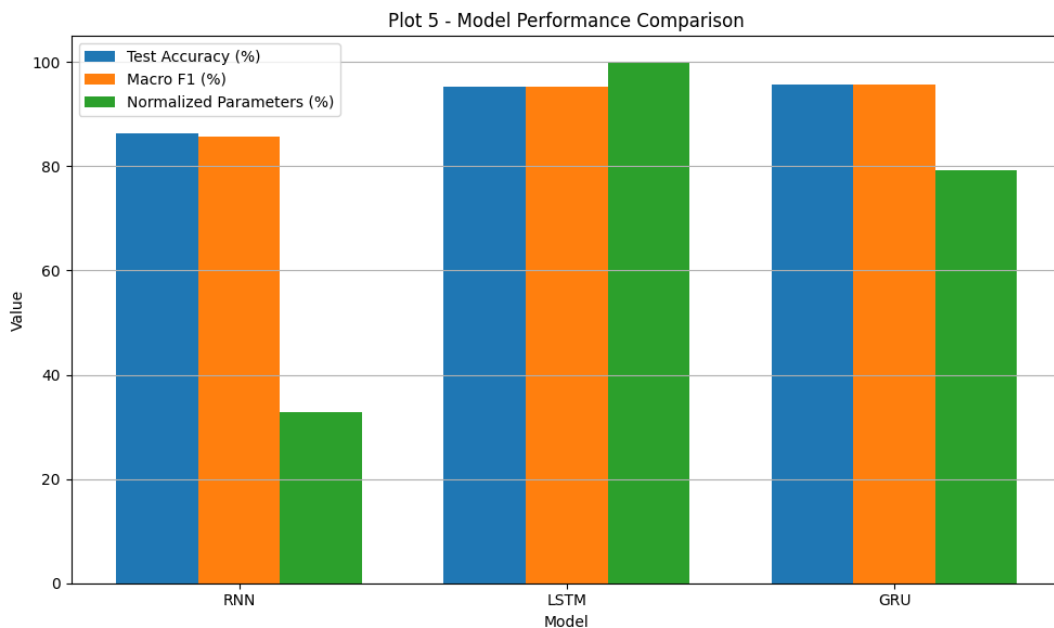


Figure 7: Model Performance Comparison

Required Inference:

- The Vanilla RNN has the simplest architecture, with only a hidden state and no gating mechanism. It has 1,974 parameters and a recorded training time of 38.05 seconds.
- The LSTM uses a cell state along with forget, input, and output gates. It has 6,006 parameters and a recorded training time of 71.18 seconds.
- The GRU uses update and reset gates with a single hidden state. It has 4,758 parameters and a recorded training time of 107.16 seconds.
- From Plot 5, the RNN achieves a test accuracy of 86.33% and a Macro F1-score of 85.76%.
- The LSTM achieves a test accuracy of 95.33% and a Macro F1-score of 95.38%.
- The GRU achieves a test accuracy of 95.67% and a Macro F1-score of 95.73%.
- The normalized parameter counts shown in Plot 5 are approximately 32.87% for RNN, 100% for LSTM, and 79.22% for GRU.
- The results demonstrate a trade-off among **Predictive Performance, Model Complexity, and Training Cost**.
- The RNN has the lowest parameter count and the shortest recorded training time, while LSTM and GRU have higher parameter counts and longer recorded training times.
- LSTM and GRU show substantially higher test accuracy and Macro F1-score than the Vanilla RNN, while their architectures contain additional gating mechanisms.

Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	82.11	95.33	95.08
64	85.17	94.67	95.39
128	79.63	94.65	96.05

Inference:

- For the Vanilla RNN, the Macro F1-score increases from 82.11% at $T = 32$ to 85.17% at $T = 64$, but decreases to 79.63% at $T = 128$.
- For the LSTM, the Macro F1-score remains relatively stable across the three sequence lengths, with values of 95.33%, 94.67%, and 94.65% for $T = 32$, 64, and 128, respectively.
- For the GRU, the Macro F1-score increases consistently from 95.08% at $T = 32$ to 95.39% at $T = 64$ and 96.05% at $T = 128$.
- The highest observed F1-score for the RNN occurs at $T = 64$ (85.17%), for the LSTM at $T = 32$ (95.33%), and for the GRU at $T = 128$ (96.05%).
- The results show that sequence length affects the three recurrent architectures differently. The RNN performance varies more noticeably with sequence length, whereas LSTM remains relatively stable and GRU shows a gradual improvement as the sequence length increases.
- Therefore, the effect of sequence length depends on the recurrent architecture and the amount of temporal information captured by the input sequence.

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

The CNN produces a feature vector of dimension

$$D = 1280.$$

Therefore, the recurrent network receives:

$$X \in \mathbb{R}^{10 \times 1280}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times 1280}.$$

LSTM Output:

Property	Value
Predicted class	WalkingWithDog
Actual class	Basketball
Confidence	95.09%

GRU Output:

Property	Value
Predicted class	WalkingWithDog
Actual class	Basketball
Confidence	90.31%

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required Observation:

The pretrained MobileNetV2 model was used as a frozen feature extractor with its classification head removed. Global Average Pooling was used to obtain a fixed-dimensional feature vector from each frame.

- Input frame size: $224 \times 224 \times 3$
- Number of frames sampled per video: 10

- CNN: MobileNetV2 pretrained on ImageNet
- CNN feature dimension:

$$D = 1280$$

- Feature sequence for one video:

$$10 \times 1280$$

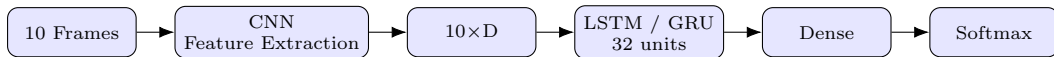
- Tensor supplied to the recurrent network for a batch of size B :

$$B \times 10 \times 1280$$

Thus, each video is represented as a sequence of 10 feature vectors, where each feature vector has 1280 dimensions. This feature sequence is then supplied as input to the recurrent classifier (LSTM/GRU).

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

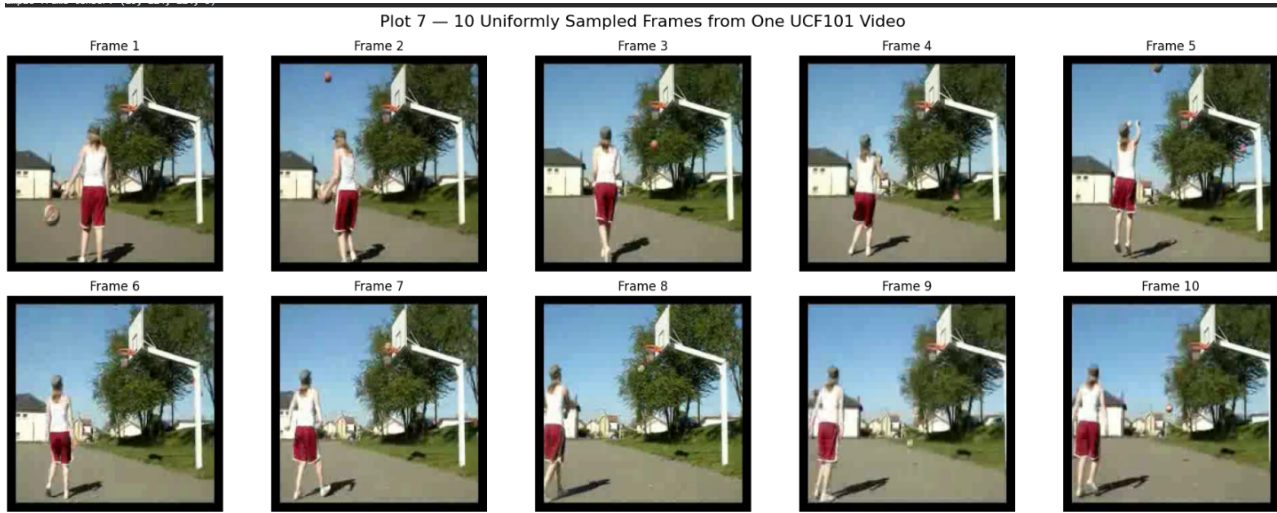


Figure 8: 10 Uniformly Sampled Frames from One UCF101 Video

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Spatial information captured by the CNN:

The CNN captures spatial information from each individual frame, such as objects, body appearance, scene/background information, and spatial arrangement of visual patterns. **Temporal information learned by the recurrent network:**

The LSTM/GRU processes the sequence of frame-level features in temporal order and learns the relationships and changes between consecutive frames. This allows the recurrent network to capture the temporal evolution of the action across the video.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

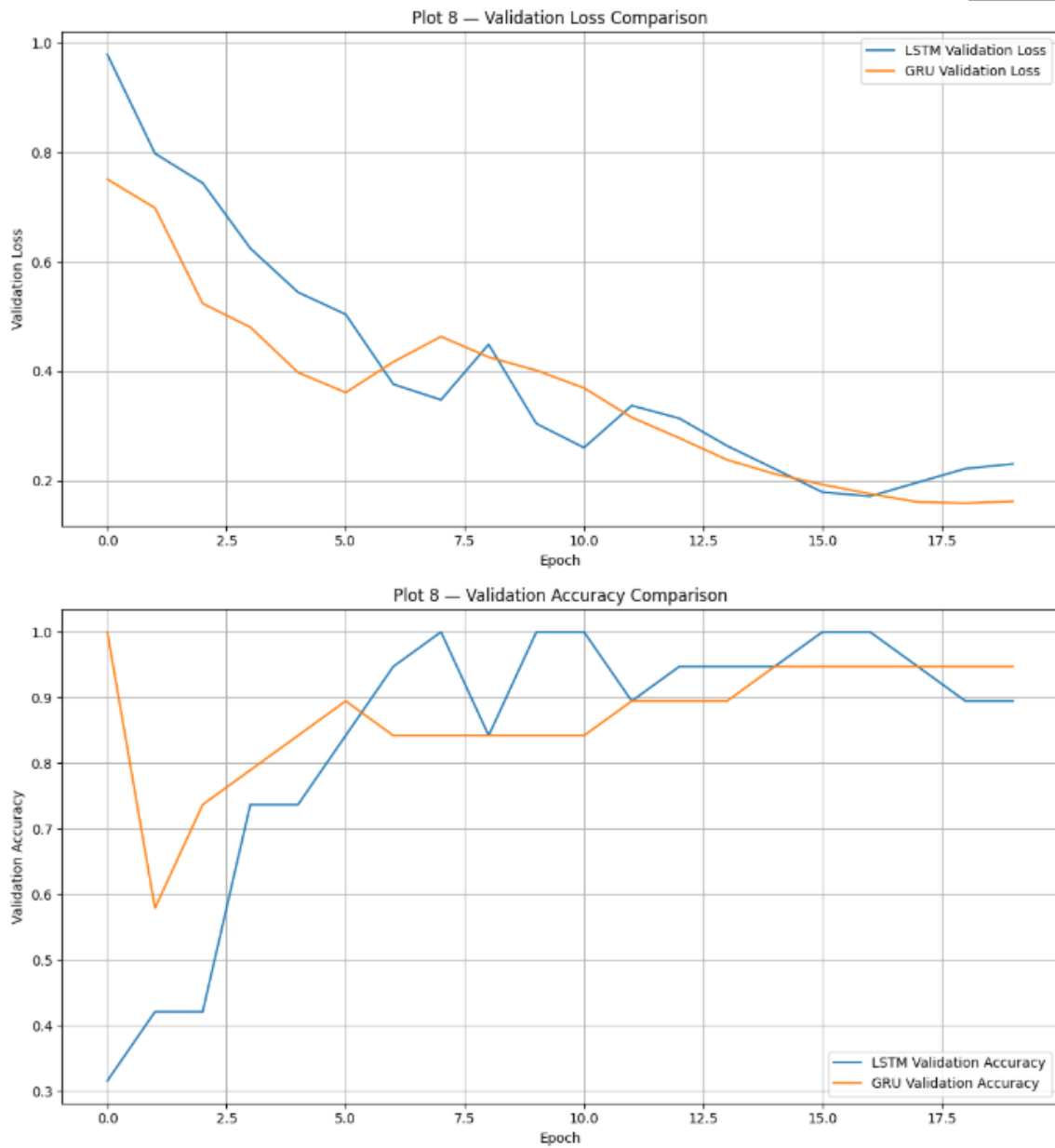


Figure 9: CNN-GRU Normalized Confusion Matrix

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

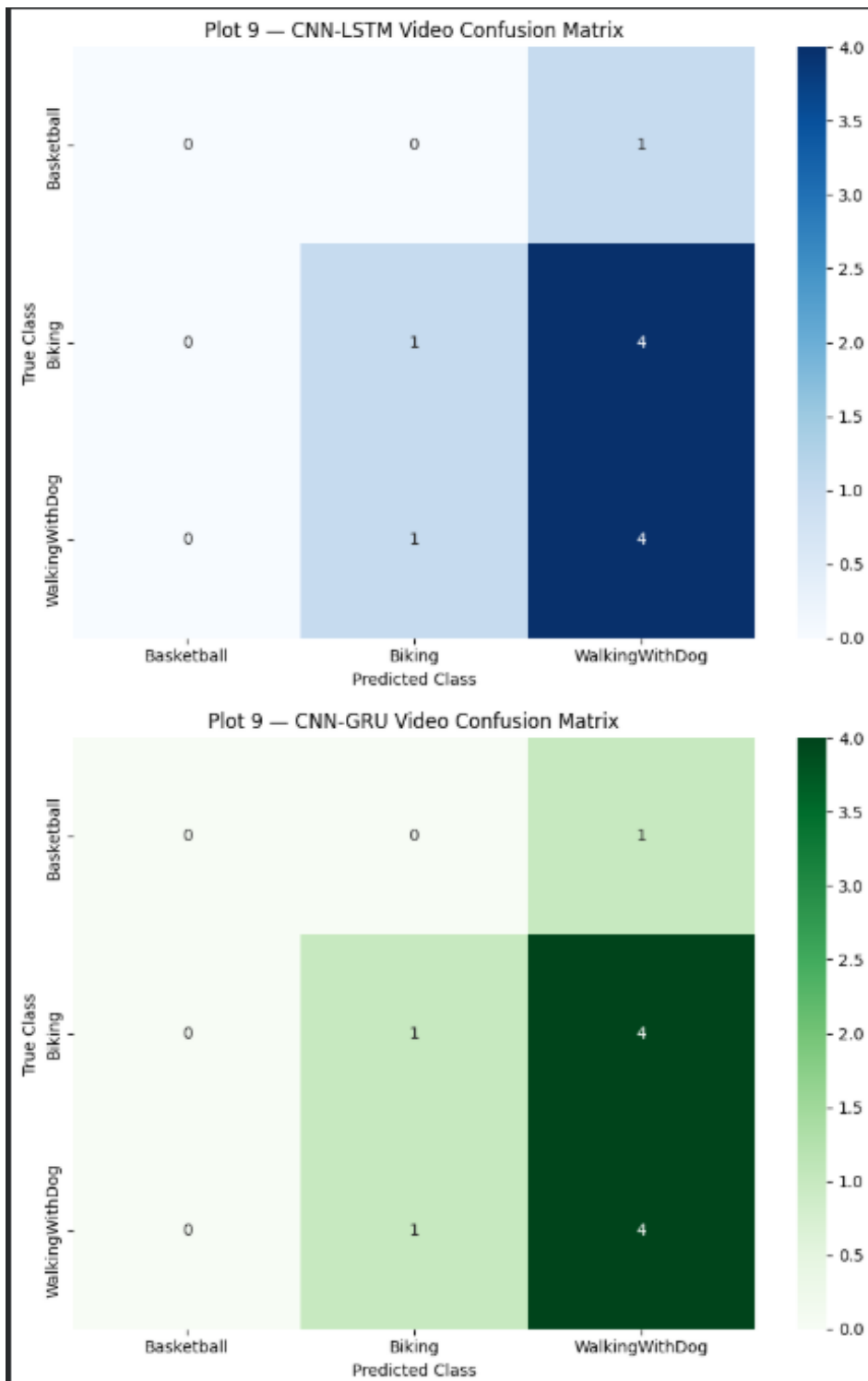


Figure 10: CNN-LSTM and CNN-GRU Video Confusion Matrices

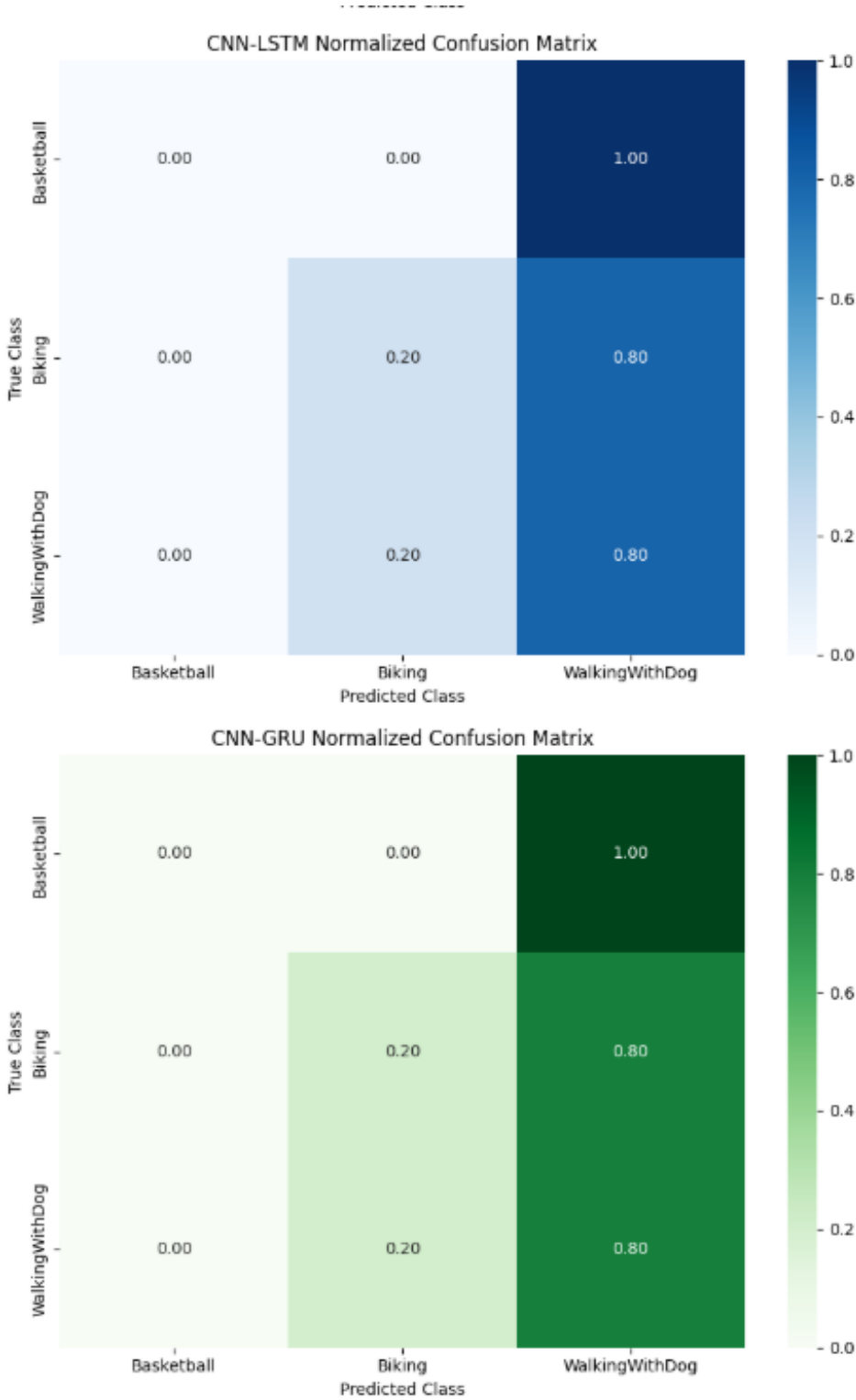


Figure 11: CNN-LSTM and CNN-GRU Normalized Confusion Matrices

Analysis:

The confusion matrices show that both CNN-LSTM and CNN-GRU have the same class-level recognition pattern. Basketball samples are all classified as WalkingWithDog, while 1 out of 5 Biking samples is correctly classified and 4 are classified as WalkingWithDog. For WalkingWithDog, 4 out of 5 samples are correctly classified and 1 is classified as Biking.

The normalized confusion matrices show the same pattern more clearly: Basketball has 0% correct recognition, Biking has 20%, and WalkingWithDog has 80%. The most frequent confusion is Biking $\rightarrow$ WalkingWithDog, with 4 misclassified samples.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3]$$

The model was tested on unseen sequences to verify whether it could reverse the input sequence correctly.

Sample	Input Sequence	Predicted Output
1	[4,2,3,7]	[7,3,2,4]
2	[7,1,8,7]	[7,8,1,7]
3	[1,8,2,8]	[8,2,8,1]
4	[3,7,6,5]	[5,6,7,3]
5	[2,2,1,9]	[9,1,2,2]

Observation:

All five test sequences were predicted correctly. The predicted output for each sequence is the reverse of the corresponding input sequence, resulting in an accuracy of 100% for the five test examples.

24. Sequence-to-Sequence Evaluation

The sequence-to-sequence model was evaluated using token accuracy, sequence accuracy, training loss, and validation loss.

Metric	Value
Token Accuracy	100.00%
Sequence Accuracy	100.00%
Training Loss	0.006343
Validation Loss	0.006993

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

For the test set,

$$\text{Token Accuracy} = \frac{3000}{3000} = 1.0000 = 100.00\%.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

For the test set,

$$\text{Sequence Accuracy} = \frac{750}{750} = 1.0000 = 100.00\%.$$

Required inference:

Sequence-level accuracy can be substantially lower than token-level accuracy because sequence accuracy is an all-or-nothing measure. A sequence is considered correct only when all of its tokens are predicted correctly.

For example, consider:

Actual = [5, 1, 8, 3]

Predicted = [5, 1, 6, 3]

Here, 3 out of 4 tokens are correct, giving a token accuracy of 75% for this sequence. However, the complete sequence is considered incorrect because one token is wrong.

Therefore, token accuracy can remain high while sequence accuracy is considerably lower. As sequence length increases, there are more opportunities for at least one token to be predicted incorrectly, making sequence accuracy a stricter evaluation metric.

Final Results:

Token Accuracy = 100.00%

Sequence Accuracy = 100.00%

Training Loss = 0.006343

Validation Loss = 0.006993

25. Consolidated Results

The consolidated results of the recurrent classification models and the CNN-based video models are presented below.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	86.33%	86.25%	85.57%	85.76%	1,974
LSTM	95.33%	95.34%	95.44%	95.38%	6,006
GRU	95.67%	95.75%	95.72%	95.73%	4,758
CNN-LSTM/GRU	45.45%	31.48%	33.33%	28.57%	168,643 / 126,723

Note: The CNN-LSTM and CNN-GRU models produced the same test-set accuracy, macro precision, macro recall, and macro F1 values. Their parameter counts were 168,643 and 126,723, respectively.

Sequence-to-Sequence Results

The sequence-to-sequence task is evaluated separately because it is a sequence-generation task rather than a single-label classification task.

Metric	Value
Token Accuracy	100.00%
Sequence Accuracy	100.00%
Training Loss	0.006343
Validation Loss	0.006993

Interpretation:

The consolidated results show the observed classification performance of the RNN, LSTM, GRU, and CNN-based recurrent models. The CNN-LSTM and CNN-GRU models use MobileNetV2-extracted frame features followed by a recurrent network for temporal processing.

For the sequence-to-sequence task, both token accuracy and sequence accuracy reached 100.00% on the evaluated test set. The training and validation losses were 0.006343 and 0.006993, respectively.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network?

A Recurrent Neural Network (RNN) is a neural network designed for sequential data. It maintains a hidden state that carries information from previous time steps and uses it along with the current input.

2. What information is represented by the hidden state?

The hidden state represents information from the current input and relevant information from previous time steps. It acts as the memory of the RNN.

3. Why is sequence order important?

Sequence order is important because the meaning of sequential data depends on the order in which the elements occur. Changing the order can change the relationship between the elements.

4. Explain the difference between a feed-forward network and an RNN.

A feed-forward network processes inputs independently without memory. An RNN maintains a hidden state, allowing information from previous time steps to influence the current output.

5. Explain Backpropagation Through Time.

Backpropagation Through Time (BPTT) is the training method used for RNNs in which the network is unrolled across time steps and gradients are propagated backward through the sequence.

6. Why can gradients vanish during BPTT?

Gradients can vanish when they are repeatedly multiplied by values smaller than one during backpropagation. The gradient becomes very small, making it difficult to learn long-term dependencies.

7. Why can gradients explode during BPTT?

Gradients can explode when they are repeatedly multiplied by values greater than one. This produces very large gradients and unstable parameter updates.

8. What are long-term dependencies?

Long-term dependencies are relationships between information that occurs far apart in a sequence. The model must retain relevant information across many time steps to learn them.

9. What is the role of the LSTM cell state?

The LSTM cell state provides a memory pathway that carries important information across time steps. The gates control which information is retained, added, or discarded.

10. What are the forget, input and output gates in LSTM?

The forget gate controls what information is discarded. The input gate controls what new information is stored. The output gate controls what information is exposed as the hidden state.

11. What are the reset and update gates in GRU?

The reset gate controls how much previous information is used when processing the current input. The update gate controls how much of the previous hidden state is retained versus new information.

12. **Compare LSTM and GRU.**

Both LSTM and GRU use gating mechanisms to learn long-term dependencies. LSTM has a separate cell state and three gates, whereas GRU uses a single hidden state with two gates.

13. **Why does GRU generally use a simpler gating mechanism than LSTM?**

GRU combines the hidden and memory representations and uses only reset and update gates. LSTM maintains a separate cell state and uses three gates.

14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**

Using the same dataset, splits, epochs, batch size, optimizer and learning rate provides a fair comparison between the architectures.

15. **What does the confusion matrix reveal that accuracy alone does not?**

A confusion matrix shows class-wise predictions and identifies which specific classes are being confused with one another.

16. **Why should macro F1-score be reported for multi-class classification?**

Macro F1-score gives equal importance to every class by calculating the F1-score for each class and averaging the results.

17. **What is the significance of sequence length?**

Sequence length determines how many time steps are processed. Longer sequences can provide more temporal information but increase computational cost.

18. **Why can a CNN be used as a feature extractor for video?**

A CNN can process individual video frames and extract useful spatial features. These features can then be given to a recurrent network for temporal analysis.

19. **What type of information is extracted by the CNN?**

The CNN extracts spatial information such as objects, body appearance, scene/background information, and spatial patterns.

20. **What type of information is learned by the recurrent network?**

The recurrent network learns temporal relationships and changes between successive frame-level features.

21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**

Each frame produces a feature vector. Arranging these vectors in temporal order creates a sequence that the LSTM or GRU can process to learn temporal relationships.

22. **Explain the encoder and decoder in sequence-to-sequence learning.**

The encoder processes the input sequence and produces a representation of it. The decoder uses this representation to generate the output sequence.

23. **What is teacher forcing?**

Teacher forcing is a training technique in which the decoder receives the actual previous target token as its input instead of its previous prediction.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy measures the fraction of individual tokens predicted correctly, whereas sequence accuracy measures the fraction of complete sequences in which every token is correct.

$$\text{Token Accuracy} = \frac{\text{Correct Tokens}}{\text{Total Tokens}}$$

$$\text{Sequence Accuracy} = \frac{\text{Completely Correct Sequences}}{\text{Total Sequences}}$$

25. **Why must the test set remain untouched during model selection?**

The test set should remain untouched so that it provides an unbiased estimate of the final model's generalization performance. Using it for model selection can lead to overly optimistic results.

28. Additional Exercises

1. **Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.**

Reducing the number of recurrent units to 16 decreases the number of trainable parameters and generally reduces training time and computational cost. Increasing the number of units to 64 increases model capacity, parameter count, and training time. The effect on accuracy and F1-score must be determined experimentally, since a larger model does not necessarily guarantee better generalization.

2. **Compare GRU and LSTM using the same number of recurrent units.**

Using the same number of recurrent units allows a fair architectural comparison. LSTM uses a cell state along with forget, input, and output gates, whereas GRU uses reset and update gates with a single hidden state.

Their accuracy, F1-score, training time, and parameter count can then be compared under identical experimental settings.

3. **Add a second recurrent layer and investigate whether performance improves.**

A second recurrent layer can learn higher-level temporal representations. The first recurrent layer must return the complete sequence so that the second recurrent layer can process it. Performance should be compared with the single-layer model using validation and test metrics. Additional layers also increase parameter count and computational cost.

4. **Compare a bidirectional LSTM with a unidirectional LSTM.**

A unidirectional LSTM processes the sequence in only the forward direction, whereas a bidirectional LSTM processes it in both forward and backward directions. The bidirectional model can use information from both past and future time steps within the input sequence, but it also requires more parameters and computation.

5. **Change the sequence length and study the effect on computational cost.**

Increasing sequence length increases the number of time steps processed by the recurrent network. This generally increases training time, memory usage, and computational cost. Shorter sequences require less computation but may contain less temporal context.

6. **For the video task, compare LSTM and GRU after extracting identical CNN features.**

The same CNN feature vectors should be supplied to both LSTM and GRU models so that the recurrent architecture is the main changing factor. The models can then be compared using accuracy, macro precision, macro recall, macro F1-score, parameter count, and training time.

7. **Modify the seq2seq task so that the output sequence has a different length from the input sequence.**

The sequence-to-sequence model can be modified so that the decoder generates a target sequence whose length differs from the encoder input length. For example,

$$\text{Input} = [2, 4, 6, 8]$$

can be mapped to a shorter output such as

$$\text{Output} = [8, 6, 4]$$

or to a longer target sequence. The decoder must generate the required number of output time steps independently of the input sequence length.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>